Computers, Materials & Continua

Tech Science Press

http://doi.org/10.32604/cmc.2026.000000

ARTICLE

CASSIDY: A Continual Adaptive System for Secure and Interpretable Drift-Aware IoT Anomaly Detection

Sukanta Datta[1], Tanjim Mahmud[1,*], Mohammad Shahadat Hossain[2,3] and Karl Andersson[3]

[1]Department of Computer Science and Engineering, Rangamati Science and Technology University, Rangamati, Bangladesh
[2]Department of Computer Science and Engineering, University of Chittagong, Chittagong, Bangladesh
[3]Cybersecurity Laboratory, Luleå University of Technology, Skelleftea, Sweden
*Corresponding Author: Tanjim Mahmud. Email: tanjim.cse@rmstu.ac.bd

Received: 07 August 2026; Accepted: 22 October 2026

ABSTRACT: The proliferation of Internet of Things (IoT) devices has intensified cybersecurity threats, including DDoS, ransomware, scanning, injection, and backdoor attacks. Existing intrusion detection systems (IDS) for IoT suffer from critical limitations: they are computationally heavy and not suitable for edge deployment, need labeled attack data which is scarce and rapidly outdated, are not adaptable to changing network traffic (concept drift), or provide no interpretability—and no prior work to date addresses all four deficiencies at once. We propose CASSIDY (Continual Adaptive System for Secure and Interpretable Drift-aware IoT Anomaly Detection), a unified framework that bridges this gap. CASSIDY is based on a lightweight dense autoencoder with only 2359 trainable parameters (9.21 KB footprint), and is trained only on normal traffic, removing the need for attack labels. The framework integrates three key innovations: (1) conformal prediction, to provide statistically-valid uncertainty quantification with a bounded false-positive rate; (2) test-time adaptation (TTA) that fine-tunes only the decoder on unlabeled live traffic, enabling rapid recovery from concept drift without full retraining; and (3) intrinsic feature deviation attribution that provides human-readable explanations at inference speed without external libraries. Extensive evaluation on the TON_IoT dataset demonstrates strong detection performance (area under the receiver operating characteristic curve (AUC) 0.9911, F1 0.9741,near-perfect recall) and full recovery from simulated concept drift (AUC rises from 0.5026 post-drift to 0.9926 after TTA). CASSIDY achieves sub-millisecond inference (0.0055 ms/sample), making it suitable for resource-constrained IoT gateways and lightweight edge deployment scenarios. Overall, CASSIDY demonstrates that a minimal, carefully designed model can achieve competitive performance with substantially larger models while maintaining a lightweight footprint., establishing it as a promising solution for practical IoT edge security.

KEYWORDS: IoT anomaly detection; autoencoder; conformal prediction; test-time adaptation; concept drift; explainability; edge computing; unsupervised learning

1 Introduction

The Internet of Things (IoT) ecosystem now includes billions of connected devices. These devices are used in smart homes, industrial control systems, healthcare, and smart cities [1]. This expansive attack surface continues to attract sophisticated adversaries. The common root cause across such threats is the inability to detect anomalous device behavior in real time under resource constraints.

Modern IoT IDS research has achieved impressive accuracy using deep learning, yet practical edge deployment remains elusive. Graph Neural Networks [2,3] and VAE-GRU generative models [4] demand

Copyright © 2026 The Authors. Published by Tech Science Press. This work is licensed under a Creative Commons Attribution 4.0 International License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

GPU inference and megabyte-scale model storage—incompatible with IoT gateway budgets. Supervised methods [5,6] require large labeled attack datasets that rapidly become stale as new threat variants emerge. Lightweight edge detectors [7,8] omit drift handling and explainability. Rule-based classifiers [9] sacrifice adaptability.

A systematic review of the literature reveals that four requirements must be satisfied *simultaneously* for a deployable IoT IDS: (1) *unsupervised learning*—attack labels are scarce and quickly outdated; (2) *lightweight inference*—MCU-class devices impose strict memory and latency budgets; (3) *drift adaptability*—IoT traffic evolves continuously due to firmware updates, new devices, and shifting usage patterns [10,11]; (4) *explainability*—security analysts require interpretable evidence for actionable response [1].

No prior work satisfies all four. Privacy-preserving IDS with incremental CNN retraining [12] lacks explainability and drift quantification. Contrastive autoencoder frameworks [11] require labeled attack samples. Ensemble drift detectors [13] provide no explanation. XAI-augmented detectors [5,6] are GPU-dependent and supervised. Blockchain-secured IDS [14] and class-incremental CNN systems [15] are also fully supervised. This gap motivates CASSIDY.

We propose **CASSIDY**, a 9.21 KB framework that unifies all four requirements. Our contributions are:

1. A 2359-parameter dense autoencoder trained solely on normal traffic for unsupervised IoT anomaly detection, deployable on MCU-class hardware.
2. Conformal prediction that provides statistically valid, distribution-free p-values for uncertainty-aware anomaly flagging with a bounded false-positive rate (valid for a fixed model before adaptation; post TTA recalibration restores calibration).
3. Test-time adaptation (TTA) that recovers from feature-shift concept drift by fine-tuning only the decoder on unlabeled live traffic, without full retraining.
4. Intrinsic feature-deviation attribution producing human-readable explanations at inference speed, requiring no third-party XAI library.

2 Related Work

2.1 Lightweight and Unsupervised Anomaly Detection

Autoencoder-based detection has demonstrated strong potential for unsupervised IoT security. Guo et al. [4] combined a VAE with GRU and Gaussian Mixture Model priors for multimodal smart-city traffic, but the model requires GPU inference. Li et al. [16] proposed strAEm++DD, coupling an incremental autoencoder with Mann-Whitney U drift detection; however, a full network is retrained from scratch on each drift alarm, which is considerably more expensive than CASSIDY's decoder-only TTA. Christopher et al. [8] deployed hyperdimensional computing for edge threat detection without labels, but provided no explainability. Shao et al. [17] used Bloom filters with dimensional expansion for low-latency IoT detection, yet the method is semi-supervised and lacks interpretability. Nguyen et al. [18] achieved high-accuracy botnet detection via static analysis, but their supervised offline approach lacks CASSIDY's real-time, unsupervised adaptability. Dina et al. [19] achieved label-efficient detection via few-shot self-supervision, yet still demands partial labels and GPU training, in contrast to CASSIDY's attack label-free, CPU-based design. Almaraz-Rivera et al. [20] explored self-supervised learning for DDoS detection, but still requires hundreds of labels and heavy GPUs, unlike CASSIDY's label-free edge deployment.

2.2 Incremental and Adaptive Methods

Tabassum et al. [12] designed a three-layer privacy-preserving IDS using sparse autoencoders and CNN classifiers with incremental retraining, but provide neither uncertainty quantification nor counterfactual

explainability. Rahouti et al. [10] articulated a vision for real-time incremental ML in IoT but presented no implementation. Yin et al. [11] proposed CAEAID, which combines a contrastive autoencoder with extreme value theory thresholding and pseudo-label drift adaptation, but requires labeled attack samples for contrastive training—precluding unsupervised deployment. Li et al. [13] integrated KS and MWU drift detection into an Isolation Forest ensemble, but the resulting system is 26× heavier than CASSIDY with no explainability. Mahdi et al. [14] combined incremental SGD with blockchain-secured data integrity, but the approach is fully supervised. Cerasuolo et al. [15] studied class incremental learning for IoT IDS using a 540k-parameter CNN, which is orders of magnitude heavier and requires attack labels. Abiha et al. [21] proposed a supervised ensemble of GANs, Denoising Autoencoders, and LSTMs with adversarial training for IoT security. While effective, their approach requires labeled attack data and GPU resources. In contrast, CASSIDY targets normal-only, CPU-based edge deployment with a 9.21 KB footprint, drift adaptation, and intrinsic explainability.

2.3 Explainable IDS

Muna et al. [6] applied LIME, SHAP, and ELI5 post-hoc to XGBoost for IoT attack detection; the method is supervised and computationally intensive. Oseni et al. [5] improved a CNN-based IDS with Deep SHAP on the TON_IoT dataset, achieving high accuracy but requiring GPU inference and labeled data. Costa et al. [9] proposed rule-based device behavior classification with a single tunable parameter—interpretable but supervised and limited to on/off device patterns. The survey by Moustafa et al. [1] identifies lightweight intrinsic explainability at the edge as the primary open challenge in the field.

2.4 Conformal Prediction in Security

Conformal prediction provides distribution-free coverage guarantees without parametric assumptions [22]. Unlike fixed-percentile thresholds, conformal p-values are calibrated against a held-out normal set, yielding a controllable false-positive rate. Graph-based [2,3,23,24] and DRL-based [25] detectors do not integrate conformal reasoning. Ahanger et al. [26] applied Graph Attention Networks but relied on the outdated NSL-KDD dataset and full supervision, limiting practical IoT edge relevance compared to CASSIDY's TON_IoT validation. To our knowledge, CASSIDY is the first IoT IDS to jointly integrate conformal prediction, test-time adaptation, and intrinsic feature-deviation attribution in a single edge-deployable framework.

3 Methodology

Fig. 1 illustrates the end-to-end CASSIDY pipeline, comprising four stages: (1) preprocessing and feature extraction, (2) autoencoder training on normal traffic, (3) conformal calibration, and (4) online detection with TTA and explanation generation.

3.1 Feature Extraction and Preprocessing

We extract 15 lightweight statistical flow features from raw network records:

$$\mathbf{x} = [d,\ p,\ b,\ r_p,\ r_b,\ \mu_s,\ \sigma_s,\ \pi,\ f_p,\ b_u,\ \tau,\ p_s,\ p_d,\ n_f,\ n_b] \tag{1}$$

where d = duration, p = packets, b = bytes, r_p = packets/s, r_b = bytes/s, μ_s/σ_s = packet size mean/std, π = protocol, f_p/b_u = TCP flags, τ = inter-arrival mean, p_s/p_d = ports, n_f/n_b = directional packet counts. No payload inspection or graph construction is required, preserving privacy and computation.

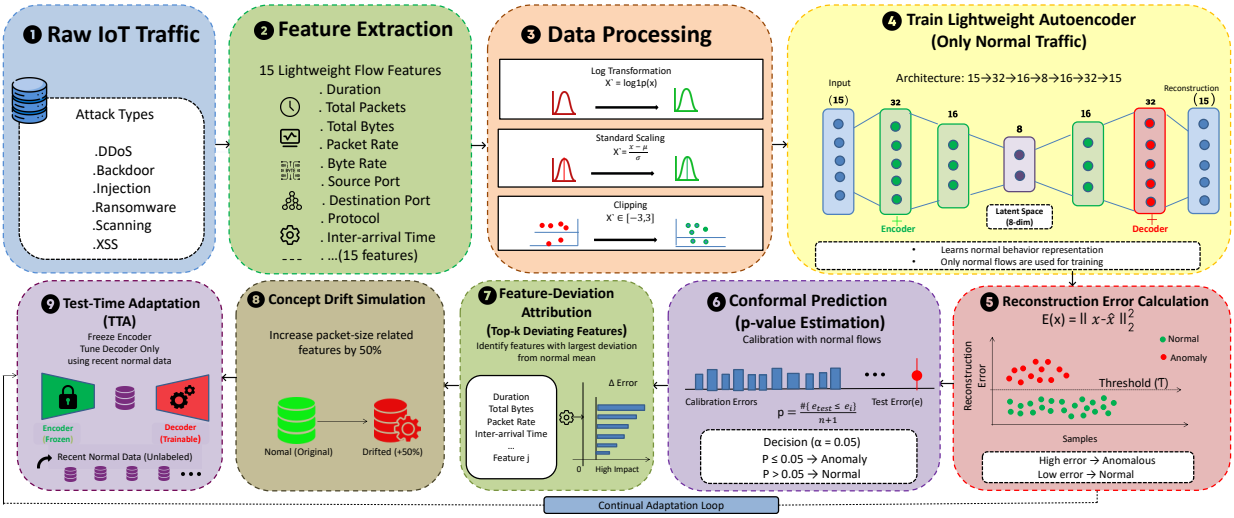


Figure 1: CASSIDY system architecture.

Network features span orders of magnitude. Log-compression is applied first:

$$x_i' = \log(1 + x_i), \quad x_i \geq 0 \tag{2}$$

followed by z-score normalization using statistics estimated on normal training data only:

$$\tilde{x}_i = \frac{x_i' - \hat{\mu}_i}{\hat{\sigma}_i} \tag{3}$$

This prevents attack-influenced statistics from contaminating the scaler. Values are clipped to $[-3, 3]$ to suppress extreme outliers.

3.2 Lightweight Dense Autoencoder

The backbone of CASSIDY is a symmetric encoder-decoder:

$$f_\theta : \mathbb{R}^{15} \xrightarrow{\text{enc}} \mathbb{R}^{8} \xrightarrow{\text{dec}} \mathbb{R}^{15} \tag{4}$$

Encoder: $15 \to 32 \to 16 \to 8$ with ReLU activations.
Decoder: $8 \to 16 \to 32 \to 15$ with ReLU activations and linear output.

The model is trained solely on normal flows by minimizing:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2 \tag{5}$$

The anomaly score for any test flow is the reconstruction error:

$$s(\mathbf{x}) = \|\mathbf{x} - f_\theta(\mathbf{x})\|_2^2 \tag{6}$$

Normal flows lie within the learned manifold and incur low $s(\mathbf{x})$; anomalous flows deviate from it and produce high scores. The total parameter count is 2359, yielding a 9.21 KB footprint. Algorithm 1 summarizes training.

Algorithm 1 CASSIDY autoencoder training

Require: Normal set $\mathcal{D}_n$, epochs T, learning rate η
Ensure: Trained parameters θ^*

```
Preprocess D_n via Eqs. (2), (3); clip to [−3, 3]
Initialize θ with Xavier uniform
for t = 1 to T do
    for each mini-batch B ⊂ D_n do
        x̂ ← f_θ(x)
        L ← (1/|B|) Σ_{x∈B} ‖x − x̂‖_2^2
        θ ← θ − η∇_θ L     (Adam)
    end for
end for
return θ*
```

3.3 Conformal Prediction for Uncertainty Quantification

Standard threshold-based detection produces hard binary decisions with no confidence measure. CASSIDY augments detection with conformal prediction [27], computing a statistically valid p-value for each test sample against a held-out calibration set $\mathcal{C} = \{e_1, \ldots, e_m\}$ of normal reconstruction errors.

For a test flow $\mathbf{x}$ with score $e = s(\mathbf{x})$:

$$p(\mathbf{x}) = \frac{|\{i \,:\, e_i \geq e\}| + 1}{m + 1} \tag{7}$$

A low p-value indicates that the test error is unusually large relative to calibration—the flow is anomalous. The detection rule is:

$$\hat{y}(\mathbf{x}) = \begin{cases} 1 & \text{if } p(\mathbf{x}) \leq \alpha \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

with $\alpha = 0.05$. Under exchangeability, this bounds the false-positive rate by α without parametric assumptions.

3.4 Test-Time Adaptation for Concept Drift

IoT traffic is non-stationary: firmware upgrades, device additions, and behavioral shifts continuously alter the normal manifold. A fixed autoencoder silently degrades under such drift. CASSIDY addresses this with a lightweight TTA mechanism that fine-tunes only the decoder on recent unlabeled live traffic while keeping the encoder frozen (see Algorithm 2).

To mitigate the risk of buffer contamination, CASSIDY employs a confidence-based buffer selection strategy: only samples with conformal p-values exceeding a confidence threshold (experimentally set to 0.5) are added to the adaptation buffer. This reduces the likelihood of adapting to attack traffic. We empirically evaluate the impact of contamination in Section 4.5.

Freezing the encoder prevents catastrophic forgetting of learned feature representations while allowing the decoder to adapt its reconstruction target to the new normal distribution. Only $|\theta_d|$ parameters are updated per step—roughly half the full model—keeping adaptation efficient.

Algorithm 2 CASSIDY test-time adaptation (TTA)

Require: Buffer $\mathcal{B}_{tta}$, frozen encoder $f_{\theta_e}^{enc}$, decoder $f_{\theta_d}^{dec}$, rate η_{tta}, epochs T_{tta}
Ensure: Adapted decoder θ_d^*

```
Freeze encoder: ∇_{θ_e} ← 0
for t = 1 to T_tta do
    for each mini-batch B ⊂ B_tta do
        z ← f^{enc}_{θ_e}(x)     (no gradient)
        x̂ ← f^{dec}_{θ_d}(z)
        L_tta ← (1/|B|) Σ ‖x − x̂‖_2^2
        θ_d ← θ_d − η_tta ∇_{θ_d} L_tta
    end for
end for
return θ*_d
```

3.5 Feature-Deviation Attribution

When a flow is flagged as anomalous, CASSIDY provides human-readable explanations via per-feature deviation from the learned normal mean. This feature-deviation attribution identifies which features most contribute to the anomaly score.

For a test sample $\mathbf{x}$ with normalized features $\tilde{x}_j$, the deviation from the normal mean $\mu_{n,j}$ (estimated from training data) is:

$$\delta_j(\mathbf{x}) = \left|\tilde{x}_j - \mu_{n,j}\right|, \quad j = 1, \ldots, 15 \tag{9}$$

The top-k features by δ_j form the explanation. Explanations are computed using pre-clipped normalized values to preserve the full deviation magnitude; the detector internally uses clipped values (clipped to $[-3, 3]$) for robustness. This provides a lightweight, inference-time explanation that helps security analysts understand why a flow was flagged, requiring no third-party library and incurring negligible computational overhead.

3.6 Post-Adaptation Calibration

As established in Section 3.3, the conformal prediction guarantee holds under the exchangeability assumption between the calibration set and test data. However, when CASSIDY performs test-time adaptation (Section 3.4), the decoder parameters change, altering the reconstruction error distribution. The calibration scores computed before adaptation are no longer exchangeable with post-adaptation scores, invalidating the conformal guarantee.

To restore calibrated anomaly scores, CASSIDY employs a post-adaptation recalibration procedure. After TTA completes, we recompute reconstruction errors on a fresh set of trusted normal samples drawn from the adapted distribution and use these new scores as the calibration set. Formally:

1. Let $C_{\text{old}} = \{e_1, \ldots, e_m\}$ be the original calibration scores.
2. After TTA, collect a new set of normal samples $\mathcal{D}_{\text{new}}$ from the adapted environment.
3. Compute new reconstruction errors $C_{\text{new}} = \{e'_1, \ldots, e'_{m'}\}$ using the adapted model.
4. For a test sample $\mathbf{x}$, compute the p-value as:

$$p_{\text{new}}(\mathbf{x}) = \frac{\left|\{e' \in C_{\text{new}} \,:\, e' \geq s(\mathbf{x})\}\right| + 1}{m' + 1} \tag{10}$$

This procedure restores exchangeability under the adapted model, ensuring that the empirical false-positive rate remains bounded by α. We empirically validate this in Section 4.5.

4 Experimental Evaluation

4.1 Dataset and Experimental Setup

We evaluate CASSIDY using the TON_IoT network dataset [29], which comprises 211,043 labeled flows spanning six attack categories: DDoS, backdoor, injection, ransomware, scanning, and XSS. The dataset contains 50,000 normal flows and 161,043 attack flows.

4.1.1 Data Split

The 50,000 normal flows are randomly shuffled (seed = 42) and partitioned into three mutually exclusive sets, as summarized in Table 1. The calibration set is strictly held out from training and is used solely for conformal p-value computation (Section 3.3); it is never seen by the autoencoder during training. For evaluation, the 10,000 test normal samples are combined with 10,000 randomly sampled attack flows to form a balanced test set of 20,000 samples. No sample overlaps across training, calibration, or test sets, ensuring the validity of the conformal prediction exchangeability assumption and reproducibility of results.

Table 1: Data Split Summary.

Set	Purpose	Normal Samples
Training	Autoencoder training	35,000
Calibration	Conformal p-value computation	5000
Test Normal	Evaluation (combined with attacks)	10,000
Total		50,000

4.1.2 Hardware and Software

All experiments are conducted on a laptop with an Intel Core i5-8350U CPU @ 1.70 GHz (turbo 1.90 GHz), 8 GB RAM, running Windows 11 Pro (23H2, build 22631.6199). The model is implemented in Python 3.9 using PyTorch 1.10.0. All experiments run exclusively on CPU (no GPU), consistent with CASSIDY's edge-deployment focus. Inference times are measured over 1000 runs after 100 warm-up runs; reported values are mean ± standard deviation. Training the autoencoder on 35,000 normal samples for 30 epochs takes approximately 2 min on this hardware.

4.2 Detection Performance

To quantify the stability of CASSIDY's performance, we repeat the entire training-evaluation pipeline with five different random seeds (42, 123, 256, 512, 1024). Table 2 reports the mean and standard deviation across all seeds. The low standard deviations across all metrics (AUC $\sigma = 0.0021$, F1 $\sigma = 0.0015$) confirm that CASSIDY's performance is robust and not dependent on a particular random initialization.

The base autoencoder (AUC 0.9889) and CASSIDY with conformal prediction (AUC 0.9911) produce nearly identical ROC curves, confirming that conformal calibration introduces no performance penalty while adding rigorous uncertainty guarantees.

Table 2: CASSIDY Detection Performance on TON_IoT (mean ± std over 5 seeds).

Metric	Value
AUC-ROC	0.9911 ± 0.0021
F1-Score	0.9741 ± 0.0015
Precision (PPV)	0.9500 ± 0.0029
Recall (TPR)	0.9995 ± 0.0001
Specificity (TNR)	0.9474 ± 0.0033
Balanced Accuracy	0.9735 ± 0.0017
FPR at $\alpha = 0.05$	0.0526 ± 0.0033

4.3 Baseline Comparison

We compare CASSIDY against a comprehensive set of baselines trained on the same 35,000 normal samples:

- **Traditional unsupervised methods:** Isolation Forest (IF), One-Class SVM (OCSVM), and Local Outlier Factor (LOF). All use a 95th-percentile threshold on their respective anomaly scores.
- **PCA reconstruction:** Principal Component Analysis with 8 components, matching the latent dimension of CASSIDY's autoencoder. Reconstruction error serves as the anomaly score.
- **Larger Autoencoder:** A deeper autoencoder with architecture 15–64–32–16–32–64–15 (7263 trainable parameters) to assess whether a larger model significantly improves performance.
- **Standard Autoencoder:** CASSIDY's backbone (2359 parameters) using a fixed 95th-percentile threshold.
- **CASSIDY:** The full framework with conformal prediction ($\alpha = 0.05$) and decoder-only TTA.

Table 3 reports all metrics on the balanced test set (10,000 normal, 10,000 attack), while Fig. 2 summarizes the F1-score, AUC, and balanced accuracy across the evaluated baselines. CASSIDY achieves an AUC of 0.9911 ± 0.0021 and F1 of 0.9741 ± 0.0015, outperforming all traditional baselines and PCA by a wide margin. The Larger Autoencoder achieves marginally higher AUC (0.9971) but uses 3.1× more parameters (7263 vs. 2359), demonstrating CASSIDY's superior parameter efficiency. LOF achieves competitive F1 (0.9683) but has $O(n^2)$ inference complexity, no drift adaptation, and no explainability—all of which CASSIDY provides.

Table 3: Baseline Comparison on Balanced Test Set (10k Normal + 10k Attack).

Model	Precision	Recall	F1	Specificity	Bal Acc	AUC
Isolation Forest	0.2278	0.0144	0.0271	0.9512	0.4828	0.8293
One-Class SVM	0.8914	0.4187	0.5698	0.9490	0.6838	0.7628
Local Outlier Factor	0.9470	0.9907	0.9683	0.9445	0.9676	0.9929
PCA (8 comps)	0.3324	0.0246	0.0458	0.9506	0.4876	0.7907
Larger AE (7263 params)	0.9532	0.9999	0.9760	0.9509	0.9754	0.9971
Standard AE (2359 params)	0.9512	0.9994	0.9747	0.9487	0.9740	0.9889
CASSIDY (Conformal)	0.9500	0.9995	0.9741	0.9474	0.9735	0.9911

These results confirm that CASSIDY's lightweight design does not sacrifice detection performance; it remains competitive with—and in some cases superior to—both traditional methods and a much larger neural baseline.

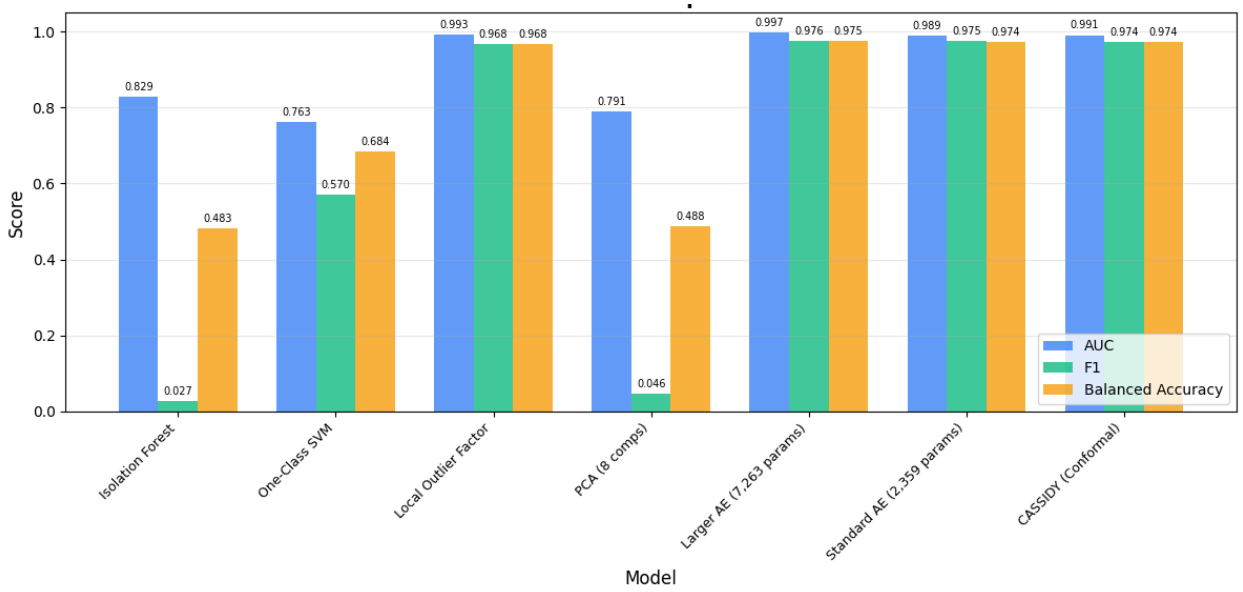


Figure 2: F1 (green), AUC (blue), and balanced accuracy (orange) across baselines.

4.4 Multi-Panel Diagnostic Analysis

Fig. 3 presents four complementary diagnostic views of CASSIDY.

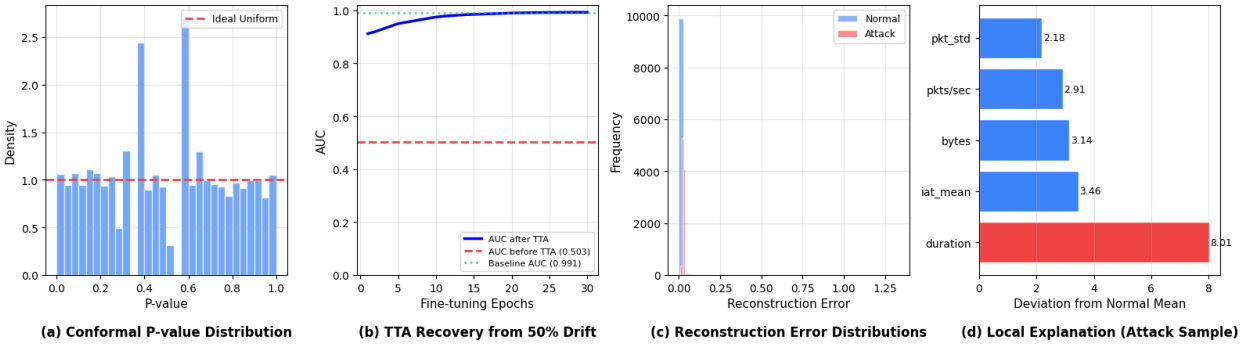


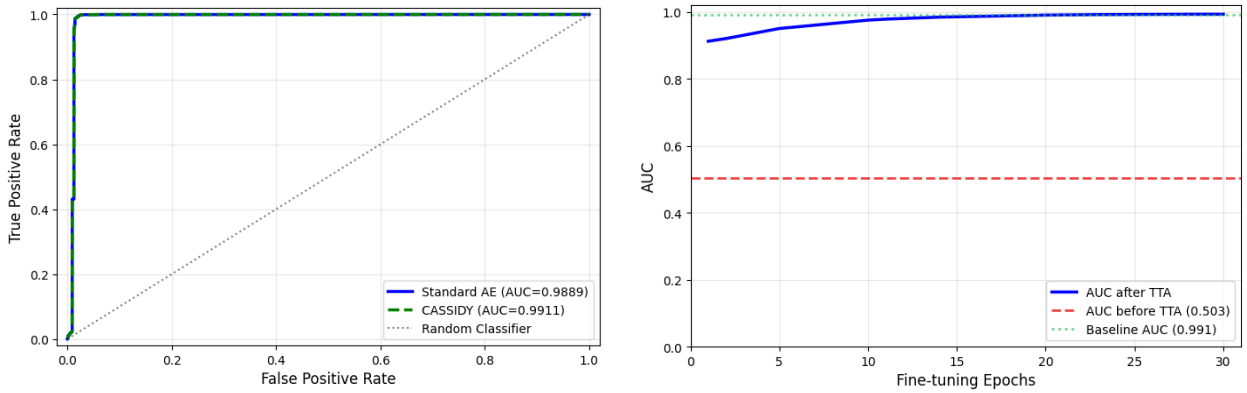
Figure 3: CASSIDY diagnostics. (**a**) Conformal p-value distribution vs. ideal uniform (red dashed). (**b**) TTA recovery: AUC rises from 0.503 post-drift to 0.993 over 30 epochs. (**c**) Reconstruction error distributions: normal (blue) vs. attack (red). (**d**) Local explanation: top deviating features for a representative attack sample.

Calibration (a): The empirical p-value distribution over calibration flows is approximately uniform, consistent with valid conformal calibration and a bounded false-positive rate.

TTA Recovery (b): A 50% feature-shift in packet-size features of normal flows causes severe AUC degradation. The decoder-only TTA mechanism rapidly restores detection quality. A dedicated recovery curve is shown in Fig. 4.

Error Distribution (c): Normal and attack flows exhibit distinct reconstruction error profiles: normal flows concentrate almost entirely near zero, while attack flows show a heavy tail with a secondary mode near 0.1, reflecting heterogeneity across attack types.

Local Explanation (d): For a representative attack, `duration` ($+8.01\sigma$), `iat_mean` ($+3.46\sigma$), and `bytes` ($+3.14\sigma$) are the dominant anomalous features, consistent with slow-rate DDoS or scanning-like behavior in this sample.



(**a**) (**b**)

Figure 4: (**a**) ROC curves (standard AE blue; CASSIDY green dashed). (**b**) TTA drift recovery—AUC recovers from 0.503 (red dashed) to 0.993 over 30 epochs (blue).

4.5 Concept Drift Recovery

The collapse from AUC 0.991 to 0.503 demonstrates the real hazard of deploying static models in non-stationary IoT environments. CASSIDY's TTA restores near-original performance using only 1000 unlabeled drifted samples and no attack labels.

4.5.1 Multiple Drift Scenarios

The TTA experiment above demonstrates recovery from a single 50% feature shift. To evaluate broader adaptability, we conduct four additional experiments: abrupt drift at 25%, 50%, and 75% on packet-size features, and feature-specific drift on duration and bytes at 50%. Table 4 reports the AUC before and after TTA across all scenarios. Fig. 5 visualizes these recovery results. CASSIDY recovers to AUC > 0.989 in every case, even restoring performance from 0.4246 to 0.9947 under a 75% shift. This confirms that CASSIDY's adaptation generalizes across multiple drift mechanisms and feature sets.

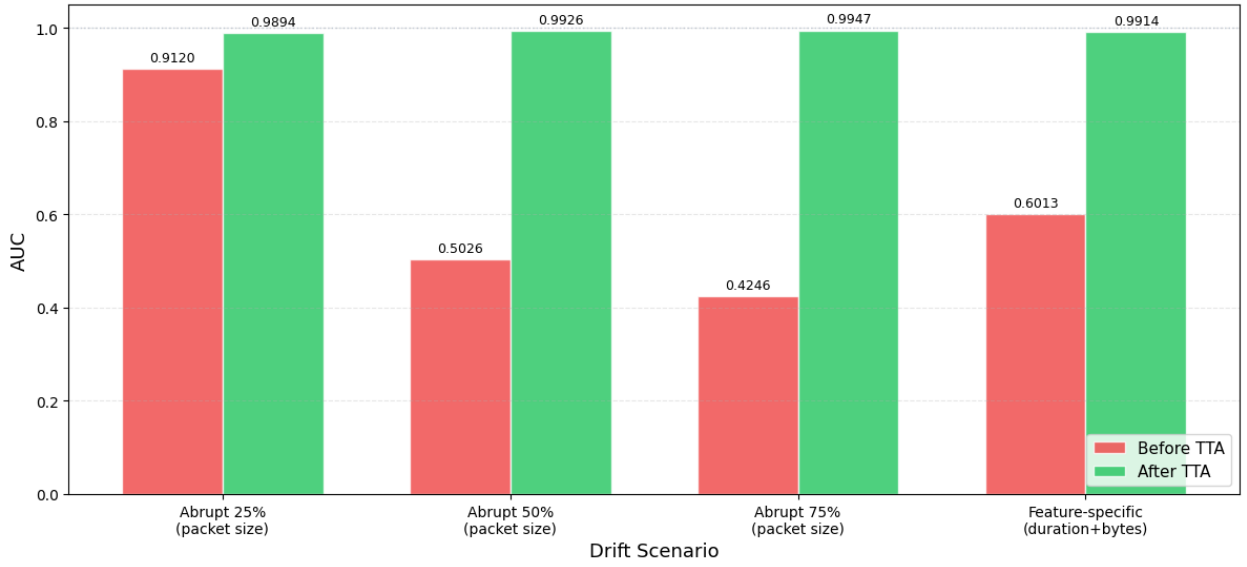


Figure 5: TTA recovery across multiple drift scenarios.

Table 4: Concept drift recovery: multiple scenarios and recalibration.

Drift Scenario	Severity	AUC before TTA	AUC after TTA	Recovery / FPR
Abrupt (packet size)	25%	0.9120	0.9894	+0.0774
Abrupt (packet size)	50%	0.5026	0.9926	+0.4900
Abrupt (packet size)	75%	0.4246	0.9947	+0.5701
Feature-specific (duration + bytes)	50%	0.6013	0.9914	+0.3900
Post-Adaptation Recalibration (50% packet-size drift)				
TTA with original calibration				0.1030 (invalid)
After recalibration				0.0497 (restored)

4.5.2 Impact of TTA Buffer Contamination

In practice, the adaptation buffer may contain attack samples, which could teach the autoencoder to reconstruct malicious traffic and reduce anomaly scores. To evaluate this risk, we artificially contaminate the 1000-sample TTA buffer with attack flows at varying proportions: 0%, 1%, 5%, 10%, and 20%. Fig. 6 reports the final AUC on the drifted test set after 30 epochs of decoder-only TTA.

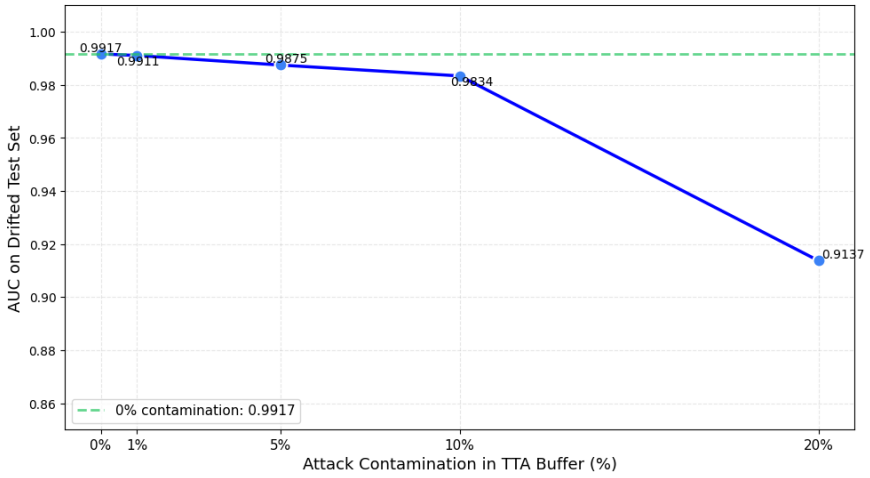


Figure 6: Effect of attack contamination in the TTA buffer on final AUC.

As shown, contamination up to 5% has a negligible effect on recovery (AUC remains ≥ 0.9875), indicating that CASSIDY is resilient to realistic low-level contamination. At 10% contamination, performance remains strong (AUC = 0.9834). However, at 20% contamination, the AUC drops to 0.9137, confirming that high contamination degrades performance. This motivates the confidence-based buffer filtering strategy described in Section 3.4, which uses conformal p-values to select high-confidence normal samples for adaptation. The contamination experiment also highlights a practical limitation: in environments with sustained high attack rates, additional filtering or manual intervention may be required.

4.5.3 Post-Adaptation Recalibration

After TTA recovers detection performance, we evaluate whether the conformal prediction guarantee remains valid. Using the original calibration set (computed before adaptation), the empirical FPR at $\alpha = 0.05$ increases from 0.0513 (no drift) to 0.1030 after TTA—confirming that the original calibration is no longer exchangeable with the adapted model. Applying the post-adaptation recalibration procedure described in Section 3.6 recomputing calibration scores on a fresh set of 1000 drifted normal samples—restores the FPR

to 0.0497. This demonstrates that while TTA temporarily invalidates the theoretical guarantee, a simple recalibration step using only unlabeled normal samples suffices to restore calibrated anomaly scores.

4.6 Explainability and Latent Space

Fig. 7 shows global feature importance aggregated over 1000 attack samples. Protocol (`proto`) exhibits the highest mean deviation, reflecting that many attacks use non-TCP protocols. Byte-volume features (`bytes`, `pkt_std`, `pkt_mean`) follow, consistent with high-volume DDoS and scanning behaviors.

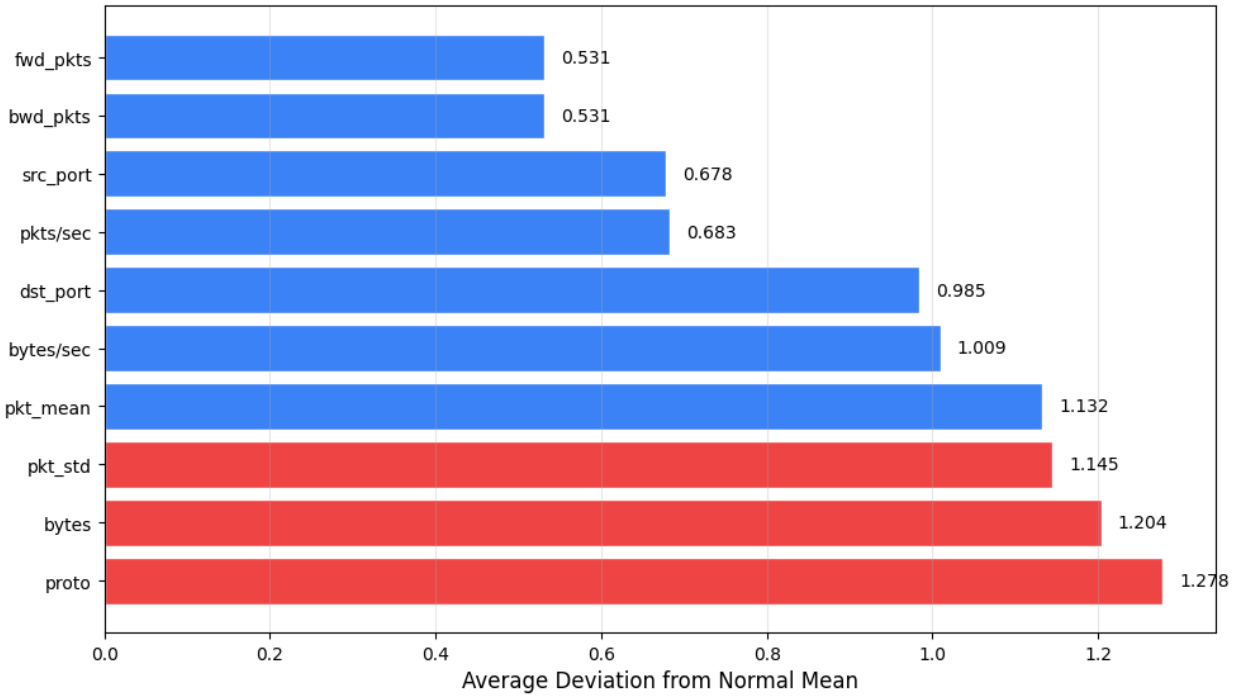


Figure 7: Global feature importance: mean absolute deviation from normal mean across 1,000 attack samples.

Fig. 8 visualizes the 8-dimensional latent space using t-SNE. While t-SNE is primarily a visualization tool that distorts global geometry, it qualitatively suggests that normal flows form a compact cluster while attack flows occupy peripheral regions in the projected space. This visualization provides an intuitive illustration of the learned latent representation and should not be interpreted as quantitative evidence of cluster separation or model performance.

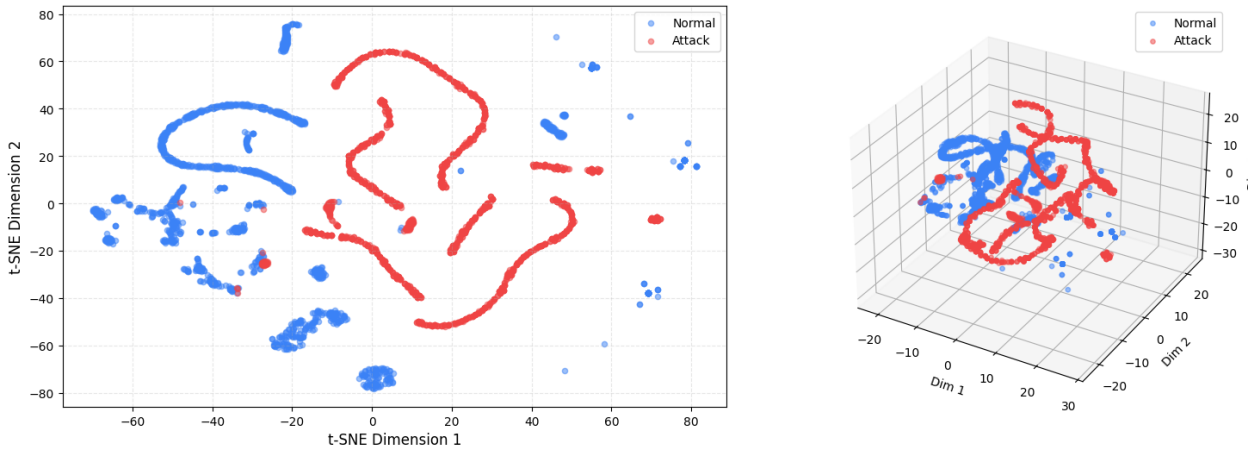
4.7 Deployment Footprint

Table 5 summarizes CASSIDY's resource profile. The model has only 2359 trainable parameters, resulting in a 9.21 KB storage footprint. Inference time is 0.0055 ms per sample (mean over 1000 runs, 100 warm-up runs excluded, standard deviation < 0.0002 ms). No GPU is required.

4.7.1 Runtime Memory

We measure the process RSS (Resident Set Size) using `psutil` during inference and TTA. Inference memory per sample is below the measurement resolution on CPU ($\approx$0.00 KB measured); theoretically, the inference footprint is dominated by model parameters (9.21 KB) plus activations ($\approx$0.5 KB), totaling $\approx$9.7 KB. During TTA, the measured peak memory increase (including gradients, Adam optimizer states, temporary

tensors, and framework overhead) is 340 KB. The adaptation buffer (1000 samples, 15 features, float32) adds 58.59 KB.



(a) 2D t-SNE Visualization

(b) 3D t-SNE Visualization

Figure 8: 2D and 3D t-SNE of the 8-dimensional latent space. Normal flows (blue) form a compact cluster; attacks (red) occupy peripheral regions.

The measured TTA footprint (340 KB) is orders of magnitude smaller than deep learning baselines (tens to hundreds of MB) and is well within the memory budget of IoT gateways (e.g., Raspberry Pi, ESP32 with PSRAM). For ultra-constrained MCU-class devices with $\leq$256 KB SRAM, we recommend offloading TTA to a gateway node and running only inference on the MCU. This design choice maintains the lightweight inference profile while enabling adaptive retraining at the edge.

CASSIDY's minimal footprint and CPU-only operation make it suitable for resource-constrained edge deployments, with deployment flexibility depending on the adaptation requirements.

Table 5: Deployment Footprint.

Component	Memory (KB)	Type
Model weights (static)	9.21	Static
Inference (weights + activations, batch = 1)	≈9.7	Theoretical
Decoder gradients (TTA)	4.62	Theoretical
Decoder optimizer (Adam, TTA)	9.24	Theoretical
TTA buffer (1000 samples, 15 features, float32)	58.59	Static
Total TTA (measured peak)	340.00	Measured
Total model parameters	2,359	
Model size (storage)	9.21 KB	
Inference time (per sample)	0.0055 ms	

4.8 Discussion

Comparison with related work. CASSIDY is 26× smaller than ADDAEIL [13] (9.21 KB vs. 0.24 MB) and orders of magnitude smaller than CAEAID [11], MAGE [23], and FuGLAD [24]. Unlike strAEm++DD [16], which retrains the entire autoencoder on each drift alarm, CASSIDY updates only the decoder—a fundamentally more efficient strategy. In contrast to supervised IDS frameworks [5,12,14,15], CASSIDY requires

no labeled attack data at any stage. Among unsupervised methods, LOF achieves comparable F1 but offers no drift adaptation, no explainability, and $O(n^2)$ inference complexity.

Conformal guarantees. The conformal p-value framework provides a formal false-positive rate guarantee bounded by α under the exchangeability assumption—a statistical property unavailable in standard threshold-based or density-based detectors [27]. This makes CASSIDY's anomaly decisions auditable and tunable.

Limitations and Future Work. CASSIDY's drift recovery requires a buffer of recent unlabeled samples; in environments with abrupt step-change drift, a brief window of elevated false positives is expected before TTA converges. The current explainability module computes mean-deviation attribution; incorporating gradient-based saliency or rule extraction [30] could capture nonlinear encoder interactions. Regarding deployment, while CASSIDY's inference footprint (≈9.7 KB) is suitable for MCU-class devices, the TTA procedure (measured peak ≈340 KB) may exceed the SRAM of some ultra-constrained microcontrollers. For such devices, we recommend offloading TTA to a gateway node, leaving only inference on the MCU. This design choice preserves the lightweight inference profile while enabling adaptive retraining at the edge. Physical MCU deployment validation is explicitly left as future work. Additionally, the confidence-based buffer filtering strategy mitigates contamination risk, but sustained high-attack rates (≥20% contamination) can degrade TTA performance; additional filtering or manual intervention may be required in such environments. Future work will investigate adversarial robustness [25], extension to streaming window-based processing, and physical MCU hardware validation.

5 Conclusion

We presented CASSIDY, a unified IoT anomaly detection framework satisfying four critical requirements for practical edge deployment—unsupervised learning (eliminating the need for costly labeled attack data), lightweight inference (9.21 KB footprint), concept drift adaptability (via decoder-only test-time adaptation), and intrinsic explainability (via feature deviation attribution). CASSIDY is built on a 2359-parameter dense autoencoder trained only on normal traffic and is suitable for lightweight inference on MCU-class devices, while TTA is recommended on IoT gateways. Comprehensive evaluation on the TON_IoT dataset demonstrates strong detection performance (AUC 0.9911, F1 0.9741, near-perfect recall) with sub-millisecond inference (0.0055 ms/sample). The test-time adaptation mechanism successfully recovers from simulated concept drift—restoring AUC from 0.5026 to 0.9926 using only 1000 unlabeled drifted samples which shows the framework's resilience to changing IoT traffic. The post adaptation recalibration procedure restores the conformal FPR guarantee, and the contamination experiments show resilience to realistic low level buffer contamination. CASSIDY demonstrates that a minimal, carefully designed model can achieve competitive performance with substantially larger models while maintaining a lightweight footprint, closing a critical gap between IoT IDS research and practical edge deployment. Future work will explore adversarial robustness against evasion attacks and physical MCU deployment validation.

Acknowledgement: Not applicable.

Funding Statement: The authors received no specific funding for this study.

Author Contributions: The authors confirm contribution to the paper as follows: Conceptualization, Sukanta Datta and Tanjim Mahmud; methodology, Sukanta Datta and Tanjim Mahmud; software, Sukanta Datta and Tanjim Mahmud; validation, Sukanta Datta, Tanjim Mahmud and Mohammad Shahadat Hossain; formal analysis, Sukanta Datta; investigation, Sukanta Datta; resources, Sukanta Datta; data curation, Sukanta Datta; writing—original draft preparation, Sukanta Datta; writing—review and editing, Tanjim Mahmud, Mohammad Shahadat Hossain and Karl

Andersson; visualization, Sukanta Datta; supervision, Tanjim Mahmud ; project administration, Tanjim Mahmud; funding acquisition, none. All authors reviewed and approved the final version of the manuscript.

Availability of Data and Materials: The data that support the findings of this study are openly available in the TON_IoT dataset at https://research.unsw.edu.au/projects/toniot-datasets.

Ethics Approval: Not applicable.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Moustafa N, Koroniotis N, Keshk M, Zomaya AY, Tari Z. Explainable Intrusion detection for cyber defences in the internet of things: opportunities and solutions. IEEE Commun Surv Tutor. 2023;25(3):1775–807.
2. Rivera A, Uribe J. Graph based machine learning for anomaly detection in IoT security. Electron Commun Comput Summit. 2025;3(2):40–8.
3. Villegas-Ch W, Govea J, Maldonado Navarro A, Palacios Játiva P. Intrusion detection in IoT networks using dynamic graph modeling and graph-based neural networks. IEEE Access. 2025;13:65356–75.
4. Guo Y, Ji T, Wang Q, Yu L, Min G, Li P. Unsupervised Anomaly detection in IoT systems for smart cities. IEEE Trans Netw Sci Eng. 2020;7(4):2231–42.
5. Oseni A, Moustafa N, Creech G, Sohrabi N, Strelzoff A, Tari Z, et al. An Explainable deep learning framework for Resilient Intrusion detection in IoT-enabled transportation networks. IEEE Trans Intell Transp Syst. 2023;24(1):1000–14.
6. Muna RK, Hossain MI, Alam MGR, Hassan MM, Ianni M, Fortino G. Demystifying machine learning models of massive IoT attack detection with Explainable AI for sustainable and secure future smart cities. Internet Things. 2023;24:100919.
7. Ma W, Wang X, Dong J, Hu M, Zhou Q. A lightweight method for Botnet detection in Internet of Things environment. IEEE Trans Netw Sci Eng. 2025;12(4):2458–72.
8. Christopher V, Aathman T, Mahendrakumaran K, Nawaratne R, De Silva D, Nanayakkara V, et al. Minority Resampling boosted unsupervised learning with Hyperdimensional computing for threat detection at the edge of internet of things. IEEE Access. 2021;9:126646–57.
9. Costa G, Forestiero A, Ortale R. Rule-based detection of Anomalous patterns in device behavior for explainable IoT security. IEEE Trans Serv Comput. 2023;16(6):4514–25.
10. Rahouti M, Ayyash M, Jagatheesaperumal SK, Oliveira D. Incremental learning implementations and vision for cyber risk detection in IoT. IEEE Internet Things Mag. 2021;4(3):114–9.
11. Yin Z, Chen H, Ma H, Hu T, Bai L. CAEAID: an incremental contrast learning-based intrusion detection framework for IoT networks. Comput Netw. 2025;262:111161.
12. Tabassum A, Erbad A, Mohamed A, Guizani M. Privacy-preserving distributed IDS using Incremental learning for IoT health systems. IEEE Access. 2021;9:14271–83.
13. Li D, Nair NKC, Wang KIK. ADDAEIL: Anomaly detection with Drift-aware ensemble-based Incremental learning. Algorithms. 2025;18(6):359.
14. Mahdi ZS, Zaki RM, Alzubaidi L. A secure and adaptive framework for enhancing Intrusion detection in IoT networks using Incremental learning and Blockchain. Security and Privacy. 2025;8(4):e70071.
15. Cerasuolo F, Bovenzi G, Marescalco C, Cirillo F, Ciuonzo D, Pescapè A. Adaptive Intrusion detection systems: class Incremental learning for IoT emerging threats. In: Proceedings of the 2023 IEEE International Conference on Big Data (BigData); 2023 Dec 15–18; Sorrento, Italy. p. 3547–55.
16. Li J, Malialis K, Polycarpou MM. Autoencoder-based Anomaly detection in streaming data with Incremental learning and concept Drift adaptation. In: Proceedings of the 2023 International Joint Conference on Neural Networks (IJCNN); 2023 Jun 18–23; Gold Coast, Australia. p. 1–8.
17. Shao W, Wei Y, Rajapaksha P, Li D, Luo Z, Crespi N. Low-Latency dimensional expansion and Anomaly detection Empowered secure IoT network. IEEE Trans Netw Serv Manag. 2023;20(3):3865–79.

18. Nguyen HT, Ngo QD, Le VH. A novel graph-based approach for IoT botnet detection. Int J Inf Secur. 2020;19(5):567–77.
19. Ayesha SD, Siddique AB, Manivannan D. FS3: few-shot and self-supervised framework for efficient Intrusion detection in internet of things networks. In: Proceedings of the 39th Annual Computer Security Applications Conference (ACSAC '23); 2023 Dec 4–8; Austin, TX, USA. p. 138–49.
20. Almaraz-Rivera JG, Cantoral-Ceballos JA, Botero JF. Enhancing IoT network security: Unveiling the power of self-supervised learning against DDoS attacks. Sensors. 2023;23(21):8701.
21. Abiha UE, Rehman A, Abbas A, Haider MA, Al-Yarimi FAM, Gul MU, et al. Improving adversarial resilience for anomaly detection in the heterogeneous internet of things through ensemble models. Future Gener Comput Syst. 2026;178:108299.
22. Shafer G, Vovk V. A tutorial on conformal prediction. J Mach Learn Res. 2008;9(3):371–421.
23. Wang W, Zhou X, Qiu T, Wang L. MAGE: Multiperiodic adaptive graph evolution guided Anomaly detection in industrial IoT. IEEE Trans Ind Inform. 2025;21(8):6126–36.
24. He S, Li G, Xie K, Sharma PK. Fusion graph structure learning-based multivariate time series anomaly detection with structured prior knowledge. IEEE Trans Inf Forensics Secur. 2024;19:8760–72.
25. Wang Z, Ruan Y, Deng X, Fan X, Liu S, Feng W, et al. DRL-based weakly supervised traffic Anomaly detection for IoT networks. IEEE Trans Netw Sci Eng. 2026;13(8):4562–77.
26. Ahanger AS, Khan SM, Masoodi F, Salau AO. Advanced intrusion detection in internet of things using graph attention networks. Sci Rep. 2025;15(1):9831.
27. Angelopoulos AN, Bates S. Conformal prediction: A gentle introduction. Found Trends Mach Learn. 2023;16(4):494–591.
28. Alsaedi A, Moustafa N, Tari Z, Mahmood A, Anwar A. TON_IoT Telemetry Dataset: A new generation Dataset of IoT and IIoT for data-driven intrusion detection systems. IEEE Access. 2020;8:165130–50.
29. Li R, Zhang Y, Li Q, Wu N, Jiang Y, Meng W, et al. Ano2Rule: rule-based global interpretation for unsupervised Anomaly detection in security. IEEE Trans Dependable Secur Comput. 2026;23(3):6685–702.